AMP 开发手册



正点原子公司名称 ：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛 ：http://www.openedv.com/forum.php

正点原子淘宝店铺 ：https://openedv.taobao.com

正点原子官方网站 ：www.alientek.com

正点原子B站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。



扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿：	正点原子	正点原子	2025.07.02

目录

前言 5
第一章 瑞芯微多核异构系统参考文档 7
1.1 多核异构系统参考文档 8
第二章 AMP 测试 9
2.1 RTT 和 HAL 调试串口 10
2.2 AP+AP 板级配置文件 12
2.2.1 测试 AP+AP 13
2.3 AP+MCU 板级配置文件 15
2.3.1 测试 AP+MCU 16

前言

ATK-DLRK3506B 开发板支持多核心异构系统，瑞芯微也提供了参考文档，许多用户看了瑞芯微的参考文档，仍然不会测试。于是笔者写了这份文档供大家参考。

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

第一章 瑞芯微多核异构系统参考文档

本章介绍瑞芯微多核异构系统参考文档路径。

1.1 多核异构系统参考文档

参考文档为网盘路径为：

08、RK 官方文档\cn\Socs\Quick-start\Rockchip_RK3506_Quick_Start_Linux_CN.pdf

08、RK 官方文档\cn\Common\AMP\Rockchip_Developer_Guide_AMP_CN.pdf。（注此文档瑞芯微并没有更新到 RK3506 相关的，但是可以作为参考。）

第二章 AMP 测试

正点原子在 2025 年 7 月份更新 SDK，已经适配了 AMP 和优化了一些功能，请大家更新 7 月份发布的 SDK，才能看到 AMP 板级配置文件。

2.1 RTT 和 HAL 调试串口

RK3506 在 HAL 和 RTT 项目中 UART4 作为调试串口，波特率为 1.5M 也就是 1500000。

那么如何确认 UART4 是哪些 IO 呢？查看<SDK>/kernel/arch/arm/boot/dts/rk3506-amp.dtsi。得知 UART4 用的是 rm_io27_uart4_tx 和 rm_io28_uart4_rx。

```
rockchip_amp: rockchip-amp {
        compatible = "rockchip,amp";
        clocks = <&cru HCLK_M0>, <&cru STCLK_M0>,
                 <&cru SCLK_UART4>, <&cru PCLK_UART4>,
                 <&cru PCLK_TIMER>, <&cru CLK_TIMER0_CH5>;

        pinctrl-names = "default";
        pinctrl-0 = <&rm_io27_uart4_tx>, <&rm_io28_uart4_rx>;

        amp-cpu-aff-maskbits = /bits/ 64 <0x0 0x1 0x1 0x2 0x2 0x4>;
        amp-irqs = /bits/ 64 <
                /* GPIO EXT */
                GIC_AMP_IRQ_CFG_ROUTE(35, 0xd0, CPU_GET_AFFINITY(0, 2))
                GIC_AMP_IRQ_CFG_ROUTE(39, 0xd0, CPU_GET_AFFINITY(0, 2))
                GIC_AMP_IRQ_CFG_ROUTE(43, 0xd0, CPU_GET_AFFINITY(0, 2))
                GIC_AMP_IRQ_CFG_ROUTE(47, 0xd0, CPU_GET_AFFINITY(0, 2))
                GIC_AMP_IRQ_CFG_ROUTE(51, 0xd0, CPU_GET_AFFINITY(0, 2))
                /* UART4 */
                GIC_AMP_IRQ_CFG_ROUTE(70, 0xd0, CPU_GET_AFFINITY(0, 2))
                /* MAILBOX */
                GIC_AMP_IRQ_CFG_ROUTE(176, 0xd0, CPU_GET_AFFINITY(0, 2))>;

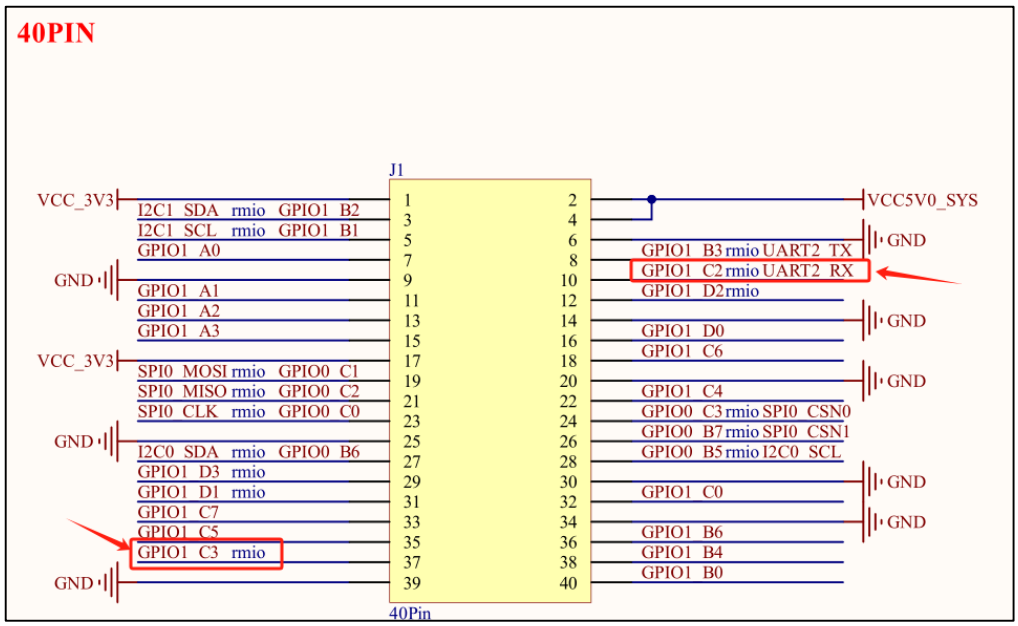
        status = "okay";
};
```

再对照<SDK>/kernel/arch/arm/boot/dts/rk3506-pinctrl-rmio.dtsi 可以知道。

```
        /omit-if-no-ref/
        rm_io28_uart4_rx: rm-io28-uart4-rx {
                rockchip,pins =
                        <1 RK_PC3 25 &pcfg_pull_up>;
        };
```

```
        };
        /omit-if-no-ref/
        rm_io27_uart4_tx: rm-io27-uart4-tx {
                rockchip,pins =
                        <1 RK_PC2 24 &pcfg_pull_none>;
        };
```

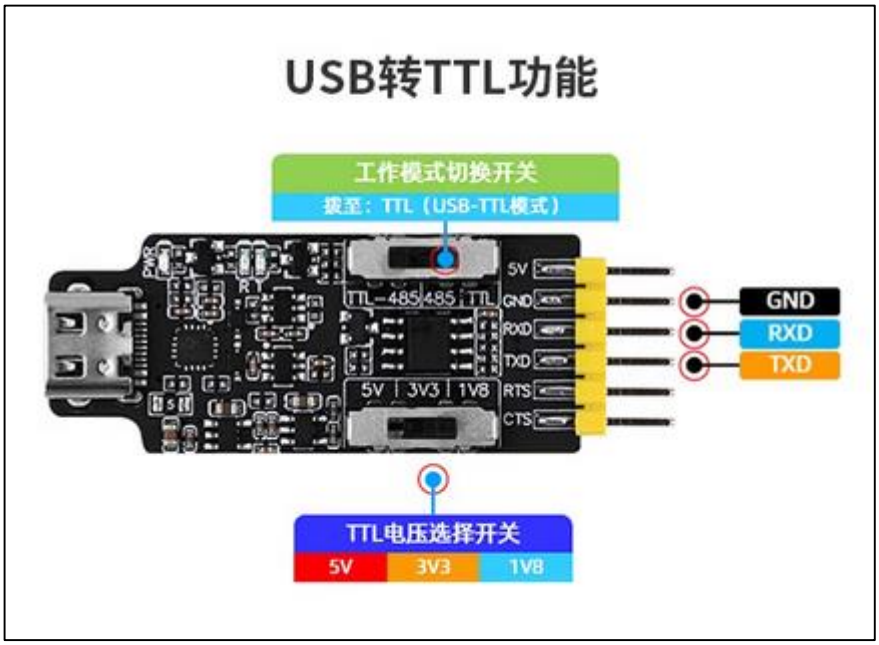
从上面我们可以知道，UART4 用的是 GPIO1_C3 和 GPIO1_C2。这两 IO 在底板上的拓展 IO 上，如果用户只有核心板，没有底板的话，就要看核心板原理图，找到对应的 IO 位置即可。



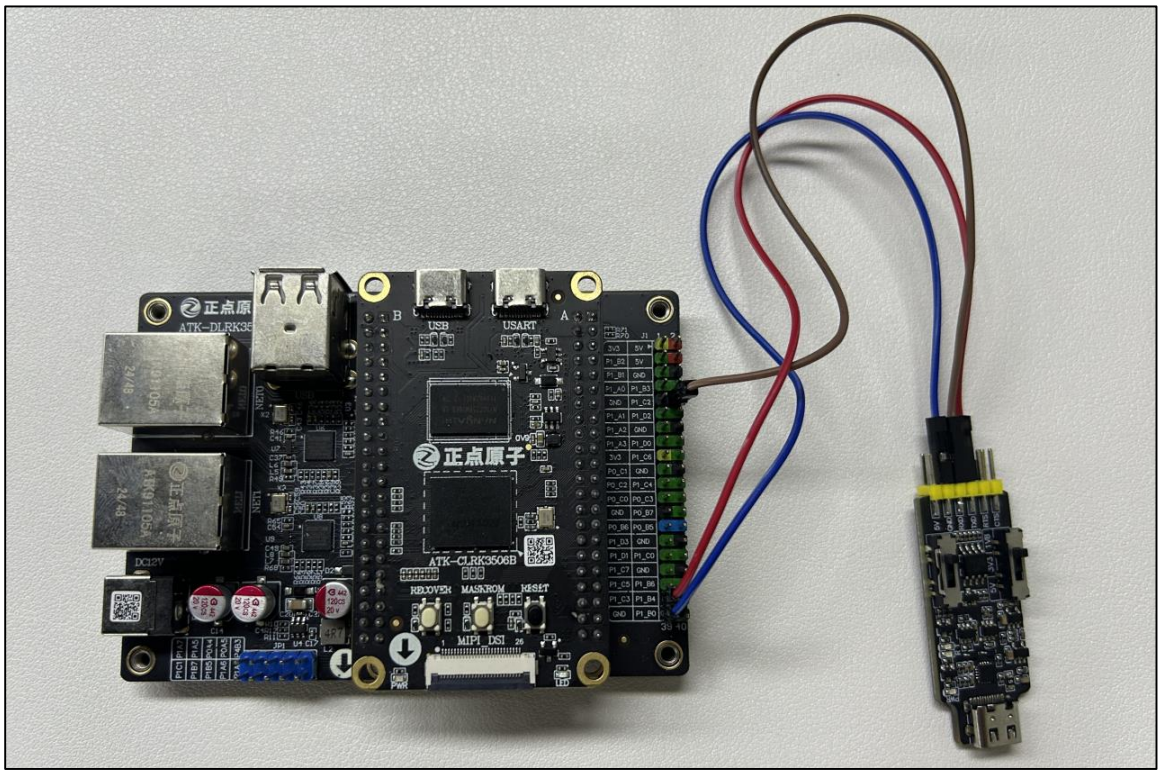
同时用户需要准备一个串口模块。如下，正点原子店铺有售卖，如果用户手上有其他串口模块亦可！



使用方法，有两个开关，如下图，两个开关拨至最右边，上方的开关拨至 TTL，下方的开关拨至 1.8V，根据开发板核心板原理图可知，RK3506B 的 GPIO1_C3 和 GPIO1_C2 支持 1.8V/3.3V。所以这里我们拨码至 1.8V。同时如下图 GND 也要用杜绑线接到开发板的 GND 上。



连接方法，串口模块的 TXD 接 GPIO1_C3，RXD 接 GPIO1_C2，GND 接开发板上的 GND。如下图。



然后用 Tpye-C 线连接到串口模块到 PC（电脑）即可。

2.2 AP+AP 板级配置文件

如果用户没有编译过 SDK，请先看网盘资料中的\10、用户手册\01、开发文档\【正点原子】ATK-DLRK3506_SDK 开发手册，里面有板级配置文件说明。

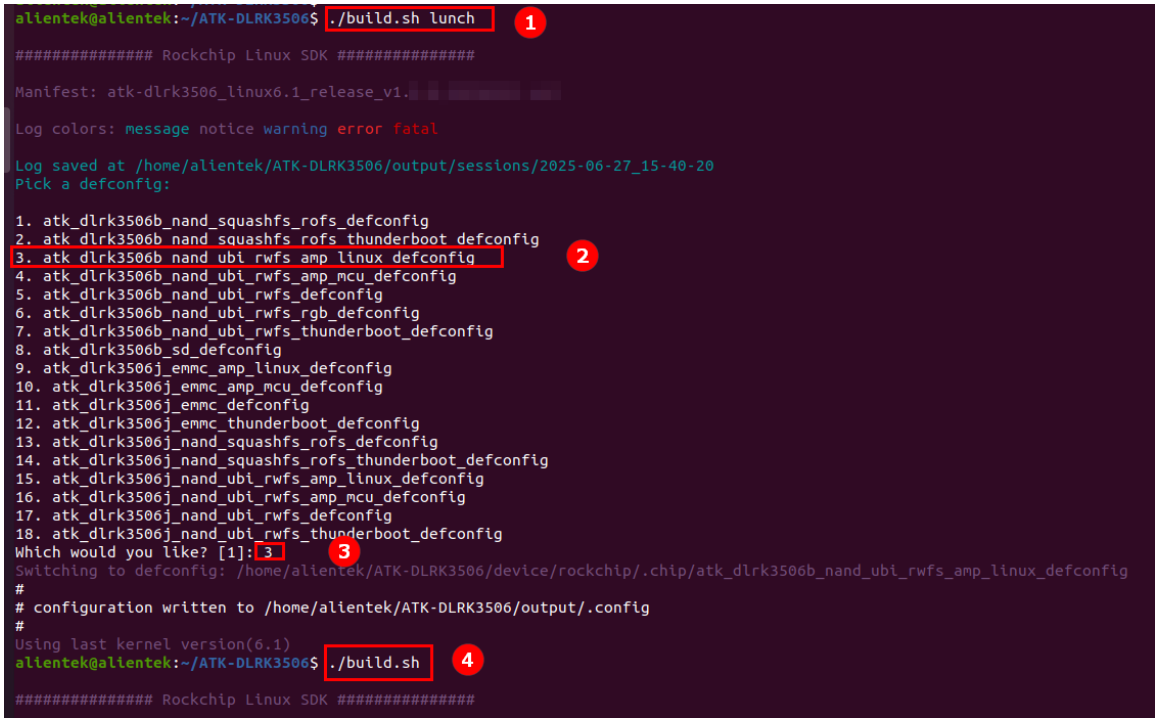
执行./build.sh lunch 选择板级配置文件。

```
./build.sh lunch
```

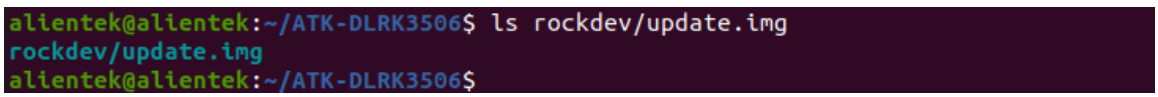
atk_dlrk3506b_nand_ubi_rwfs_amp_linux_defconfig 对应的是序号 3（请用户再次确认，防止新 SDK 导致序号发生改变！），那么我们输入 3。那么编译出来的 update.img 就是 AP（Linux）+AP（实时系统）。带“amp_linux”的为 AP（Linux）+AP（实时系统）。

如下图输入 3，选择板级配置文件，然后执行./build.sh 编译即可。

```
./build.sh
```



等待编译完成，将生成在<SDK>/rockdev/update.img 生成镜像文件，然后烧录到开发板即可。



2.2.1 测试 AP+AP

atk_dlrk3506b_nand_ubi_rwfs_amp_linux_defconfig 此配置设置了两个 A7 跑 Linux，第三个 A7，也就是 CPU2 跑 RTT（实时操作系统）。

详细请看<SDK>/device/rockchip/rk3506/atk_dlrk3506b_nand_ubi_rwfs_amp_linux_defconfig 和<SDK>/device/rockchip/rk3506/amp_linux.its。

此配置编译的 RTT 源码位于<SDK>/rtos/bsp/rockchip/rk3506-32 下。

在上一步，我们已经烧写镜像到开发板，现在我们来测试 AP+AP。

启动系统后，Uboot 打印了这样的信息，可以看到 amp 镜像在 Uboot 中加载，在 CPU 2 运行。后面的信息为内存分配提示，默认即可。

```
## Loading loadables from FIT Image at 1c803700 ...
   Trying 'amp2' loadables subimage
     Description:  rtos-core2
     Type:         Firmware
     Compression:  uncompressed
     Data Start:   0x1c804500
     Data Size:    104992 Bytes = 102.5 KiB
     Architecture: ARM
     Load Address: 0x03e00000
     Hash algo:    sha256
     Hash value:   a7a9fe2e25b6e5a845485d36edb7650fc9cc085c1032412c28078036a1927f9c
   Verifying Hash Integrity ... sha256+ OK
   Loading loadables from 0x1c804500 to 0x03e00000
AMP: Brought up cpu[f02] with state 0x10, entry 0x03e00000 ...OK
I/TC: Secondary CPU 2 initializing
I/TC: Secondary CPU 2 switching to normal world boot
Net:   eth0: ethernet@ff4c8000, eth1: ethernet@ff4d0000
Hit key to stop autoboot('CTRL+C'):  0
## Booting FIT Image at 0x1be466c0 with size 0x004b1800
Fdt Ramdisk skip relocation
Sysmem Warn: kernel 'reserved-memory' "amp@3e00000"(0x03e00000 - 0x03f00000) is overlap with "rtos-core2" (0x03e00000 - 0x03e19a20)

sysmem_dump_all:
    ------------------------------------------------------------------
    memory.rgn[0].addr     = 0x00000000 - 0x20000000 (size: 0x20000000)

    memory.total           = 0x20000000 (512 MiB. 0 KiB)
    ------------------------------------------------------------------
    allocated.rgn[0].name  = "UBOOT"
                    .addr  = 0x1c4f8ee0 - 0x1fa00000 (size: 0x03507120)
    allocated.rgn[1].name  = "STACK"
                    .addr  = 0x1c2f8ee0 - 0x1c4f8ee0 (size: 0x00200000)
    allocated.rgn[2].name  = "rtos-core2"
                    .addr  = 0x03e00000 - 0x03e19a20 (size: 0x00019a20)
    allocated.rgn[3].name  = "FIT"
                    .addr  = 0x1be466c0 - 0x1c2f7ec4 (size: 0x004b1804)
    allocated.rgn[4].name  = "FDT"
                    .addr  = 0x00063000 - 0x0006da04 (size: 0x0000aa04)
    allocated.rgn[5].name  = "KERNEL"
                    .addr  = 0x01100000 - 0x015a6604 (size: 0x004a6604)

    kmem-resv.rgn[0].name  = "ramoops@83000"
                    .addr  = 0x00083000 - 0x000b0000 (size: 0x0002d000)
    kmem-resv.rgn[1].name  = "amp-shmem@3b00000"
                    .addr  = 0x03b00000 - 0x03c00000 (size: 0x00100000)
    kmem-resv.rgn[2].name  = "rpmsg@3c00000"
                    .addr  = 0x03c00000 - 0x03d00000 (size: 0x00100000)
    kmem-resv.rgn[3].name  = "rpmsg-dma@3d00000"
                    .addr  = 0x03d00000 - 0x03e00000 (size: 0x00100000)
    kmem-resv.rgn[4].name  = "amp@3e00000"
                    .addr  = 0x03e00000 - 0x03f00000 (size: 0x00100000)

    framework malloc_r     =  16 MiB
    framework malloc_f     =   0 KiB

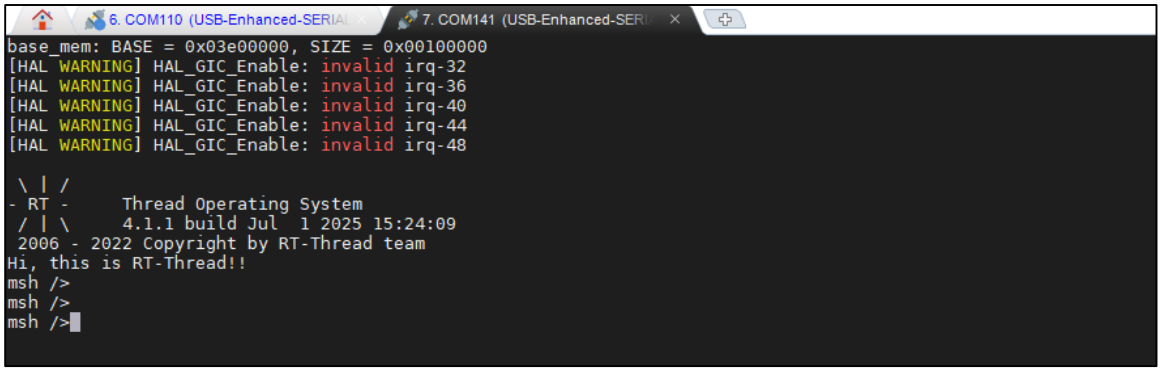
    allocated.total        = 0x044b034c (68 MiB. 704 KiB)
```

在 Linux 系统中查看 CPU 情况如下，使用 top 指令查看。可以看到下图只有 CPU0 和 CPU1 了，CPU2 已经跑 RTT 了。那么如何知道 CPU2 跑了 RTT 呢？

```
top - 08:17:18 up 17 min,  0 users,  load average: 0.07, 0.04, 0.05
Tasks:  96 total,   1 running,  95 sleeping,   0 stopped,   0 zombie
%Cpu0  :   2.0/0.7     3[          ]     %Cpu1  :   0.0/0.7     1[          ]
GiB Mem : 16.4/0.4      [          ]
GiB Swap:  0.0/0.0      [          ]

  PID USER      PR  NI    VIRT    RES  %CPU  %MEM     TIME+ S COMMAND
    1 root      20   0    2.9m   0.8m   0.0   0.2   0:00.53 S init
  236 root      20   0    2.9m   0.6m   0.0   0.1   0:00.06 S  `- /sbin/syslog+
  240 root      20   0    2.9m   0.6m   0.0   0.1   0:00.03 S  `- /sbin/klogd +
  312 root      20   0   15.4m   1.4m   0.0   0.3   0:00.35 S  `- /sbin/udevd +
  365 root      20   0   10.5m   0.5m   0.0   0.1   0:00.27 S  `- /usr/sbin/ir+
  379 dbus      20   0    2.8m   1.2m   0.0   0.3   0:00.04 S  `- dbus-daemon +
  389 root      20   0    2.0m   0.8m   0.0   0.2   0:00.01 S  `- /usr/sbin/rp+
  411 root      20   0    4.9m   2.1m   0.0   0.5   0:00.12 S  `- /usr/libexec+
  470 dhcpcd    20   0    2.5m   1.2m   0.0   0.3   0:00.01 S  `- dhcpcd: [man+
  474 root      20   0    2.5m   1.5m   0.0   0.3   0:00.15 S      `- dhcpcd: +
  475 dhcpcd    20   0    2.2m   0.2m   0.0   0.0   0:00.00 S      `- dhcpcd: +
  476 dhcpcd    20   0    2.2m   0.2m   0.0   0.0   0:00.00 S      `- dhcpcd: +
  520 root      20   0    4.9m   2.3m   0.0   0.5   0:00.17 S  `- /usr/sbin/co+
  526 ntp       20   0    8.8m   1.9m   0.0   0.4   0:00.37 S  `- /usr/sbin/nt+
  532 root      20   0   33.8m  26.0m   0.0   5.9   0:03.61 S  `- /usr/bin/wes+
  546 root      20   0   10.2m   4.8m   0.0   1.1   0:00.37 S      `- /usr/lib+
  547 root      20   0   11.2m   6.6m   0.0   1.5   0:00.50 S      `- /usr/lib+
```

在 2.1 小节连接 UART4 就可以看到打印信息，这样就可以知道 RTT 跑起来了啊。在串口终端设置 1500000 波特率再打开串口，然后重启开发板。UART4 会打印如下信息。说明 RTT 已经在 CPU2 跑起来了！



最后我们如果要开发 RTT，如用户想要实现某些功能如使用 rpms 核间通信，还需要参考此文档 1.1 小节的参考文档。

2.3 AP+MCU 板级配置文件

如果用户没有编译过 SDK，请先看网盘资料中的\10、用户手册\01、开发文档\【正点原子】ATK-DLRK3506_SDK 开发手册，里面有板级配置文件说明。

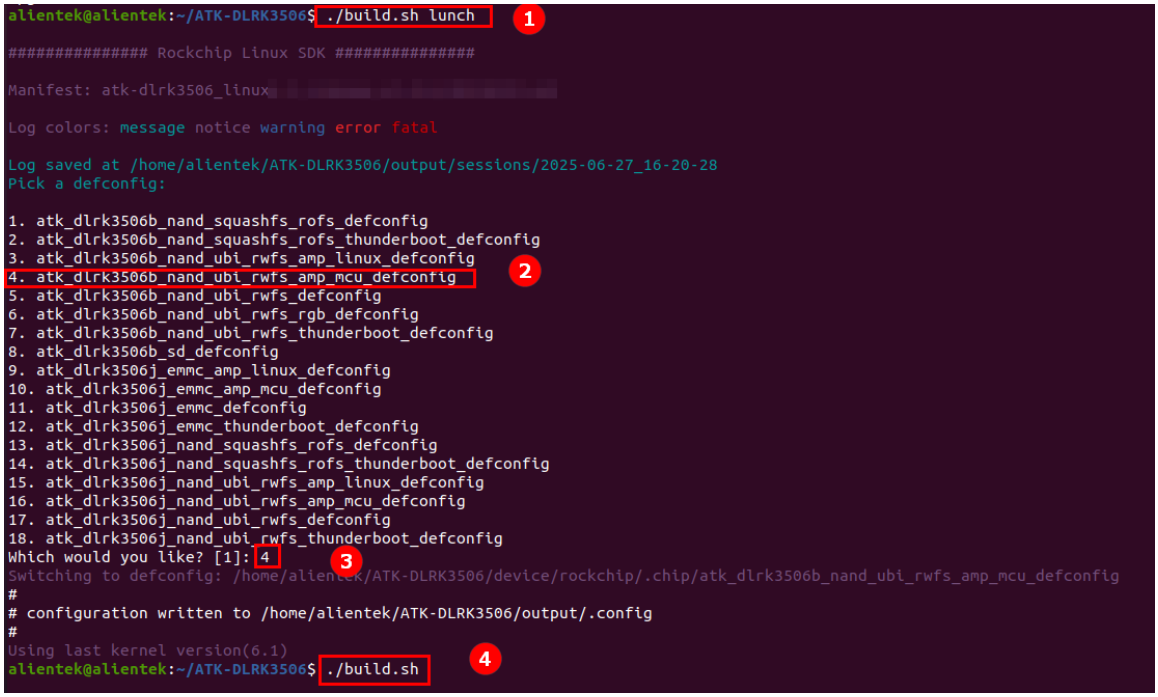
执行./build.sh lunch 选择板级配置文件。

```
./build.sh lunch
```

atk_dlrk3506b_nand_ubi_rwfs_amp_mcu_defconfig 对应的是序号 4（请用户再次确认，防止新 SDK 导致序号发生改变！），那么我们输入 4（。那么编译出来的 update.img 就是 AP（Linux）+MCU（HAL）。带“amp_mcu”的为 AP（Linux）+MCU（HAL）。

如下图输入 3，选择板级配置文件，然后执行./build.sh 编译即可。

```
./build.sh
```



等待编译完成，将生成在<SDK>/rockdev/update.img 生成镜像文件，然后烧录到开发板即可。

```
alientek@alientek:~/ATK-DLRK3506$ ls rockdev/update.img
rockdev/update.img
alientek@alientek:~/ATK-DLRK3506$
```

2.3.1 测试 AP+MCU

atk_dlrk3506b_nand_ubi_rwfs_amp_mcu_defconfig 此配置设置了三个 A7 跑 Linux，也就是 M0 跑 Bare-Metal，目前 RK3506 MCU 只支持 Bare-Metal。

详细请看<SDK>/device/rockchip/rk3506/atk_dlrk3506b_nand_ubi_rwfs_amp_mcu_defconfig 和<SDK>/device/rockchip/rk3506/amp_mcu.its。

此配置编译的 Bare-Metal 源码位于<SDK>/hal/project/rk3506-mcu 下。

在上一步，我们已经烧写镜像到开发板，现在我们来测试 AP+MCU。

启动系统后，Uboot 打印了这样的信息，可以看到 amp 镜像在 Uboot 中加载，打印了如下信息。

```
final DSI-Link bandwidth: 849 Mbps x 2
CLK: (arm clk use pvtpll, rate = 1200M)
  gpll 1500000 KHz
  v0pll 1179648 KHz
  v1pll 903168 KHz
  clk_gpll_div 187500 KHz
  clk_gpll_div_100m 93750 KHz
  clk_v0pll_div 147456 KHz
  clk_v1pll_div 129024 KHz
  aclk_bus_root 187500 KHz
  hclk_bus_root 147456 KHz
  pclk_bus_root 93750 KHz
  aclk_hsperi_root 147456 KHz
  hclk_hsperi_root 93750 KHz
## Loading loadables from FIT Image at 1c804080 ...
   Trying 'mcu' loadables subimage
     Description:  mcu
     Type:         Standalone Program
     Compression:  uncompressed
     Data Start:   0x1c804e80
     Data Size:    24064 Bytes = 23.5 KiB
     Architecture: ARM
     Load Address: 0xfff84000
     Entry Point:  unavailable
     Hash algo:    sha256
     Hash value:   d549e9409603b65fa239d46198e780a176bc9baf293abcb604eccf61a329b615
   Verifying Hash Integrity ... sha256+ OK
   Loading loadables from 0x1c804e80 to 0xfff84000
Handle standalone: 'mcu' at 0xfff84000 ...OK
Net:   eth0: ethernet@ff4c8000, eth1: ethernet@ff4d0000
Hit key to stop autoboot('CTRL+C'):  0
## Booting FIT Image at 0x1be466c0 with size 0x004b1800
Fdt Ramdisk skip relocation
## Loading kernel from FIT Image at 1be466c0 ...
   Using 'conf' configuration
```

同理，进入系统后，我们先观察 AP 的运行情况。看看是不是 3 个 CPU 在跑。

```
top - 08:02:07 up 2 min,  0 users,  load average: 0.44, 0.29, 0.11
Tasks: 112 total,   1 running, 111 sleeping,   0 stopped,   0 zombie
%Cpu0  :   0.0/0.0     0[          ]     %Cpu1  :   0.7/0.7     1[          ]
%Cpu2  :   2.0/1.3     3[          ]
GiB Mem : 16.6/0.4      [          ]
GiB Swap:  0.0/0.0      [          ]

  PID USER      PR  NI    VIRT    RES  %CPU  %MEM     TIME+ S COMMAND
    1 root      20   0    2.9m   0.8m   0.0   0.2   0:05.50 S init
  246 root      20   0    2.9m   0.6m   0.0   0.1   0:00.08 S  `- /sbin/syslog+
  250 root      20   0    2.9m   0.6m   0.0   0.1   0:00.02 S  `- /sbin/klogd +
  324 root      20   0   15.4m   1.5m   0.0   0.3   0:00.47 S  `- /sbin/udevd +
  377 root      20   0   10.5m   0.5m   0.0   0.1   0:00.09 S  `- /usr/sbin/ir+
  403 dbus      20   0    2.8m   1.2m   0.0   0.3   0:00.05 S  `- dbus-daemon +
  415 root      20   0    2.0m   0.8m   0.0   0.2   0:00.01 S  `- /usr/sbin/rp+
  434 root      20   0    4.9m   2.2m   0.0   0.5   0:00.22 S  `- /usr/libexec+
  467 dhcpcd    20   0    2.5m   1.2m   0.0   0.3   0:00.02 S  `- dhcpcd: [man+
  468 root      20   0    2.5m   1.4m   0.0   0.3   0:00.16 S     `- dhcpcd: +
```

在 2.1 小节连接 UART4 就可以看到打印信息，这样就可以知道 MCU 跑起来了啊。在串口终端设置 1500000 波特率再打开串口，然后重启开发板。UART4 会打印如下信息。说明 MCU 跑起来了！

```
[HAL INFO] Hello RK3506 mcu
```